Operating Systems - Assignment 9: xv6 Memory Management Report

Built on mit-pdos/xv6-public (x86), run and tested under QEMU. Q1, Q2 and Q3 are kept in separate source trees (q1/, q2/, q3/) because lazy allocation in Q2 would change what the Q1 test shows. Every change is commented with "Q1" or "Q2" in the source; changes.diff in each folder shows the full diff against upstream. Screenshots in screenshots/ are captures of the QEMU console while the tests ran.

Question 1 - vmprint()

Changed: vm.c (new vmprint()), sysproc.c (sys_vmprint wrapper), syscall.c/syscall.h/usys.S/user.h/defs.h (register syscall 22), Makefile (pttest). vmprint() walks the page directory of the calling process over the user entries only (PDX < PDX(KERNBASE)), the same loop shape as freevm(). For each present PTE it prints VA, PA and the P/W/U flags, then the number of valid user pages and the number of page-table pages (second-level tables, plus the directory).
Output: pttest shows 3 valid pages at start-up (code/data, guard page with U=0, stack) and 7 after sbrk(4*4096): exactly four new pages (0x3000-0x6000), all P=1 W=1 U=1. Page-table pages stay at 1 + directory because the new pages fit in the same 4 MB region. Screenshot: q1_pttest.png.

Question 2 - Lazy allocation

Changed: sysproc.c (sys_sbrk only raises sz for n > 0, plus sys_getfaults), trap.c (T_PGFLT handler), vm.c (mappages made non-static, copyuvm skips never-allocated pages instead of panicking, deallocuvm skips them), proc.h (pgfaults field), proc.c and exec.c (reset counter for a new process image), syscall files, Makefile (faulttest). The handler reads rcr2(); if the page is not present and the address is below p->sz it kalloc()s a page, zeroes it, maps it with PTE_W|PTE_U, increments pgfaults and returns to retry the instruction. Otherwise (address at or above sz, or a protection fault on a present page) it prints the pid and address and sets killed.
Output: faulttest calls sbrk(32*4096): 0 faults. Writing to 5 different pages gives 1,2,3,4,5 faults, final count 5 (PASS). A forked child that touches an address above sz is killed with "page fault at invalid address 0x2d000", and the parent keeps running. Screenshot: q2_faulttest.png.

Question 3 - FIFO and LRU

Added: pagerepl.c (user program, no kernel change) and its Makefile entry. simulate() keeps one stamp per frame: the load time for FIFO and the last-use time for LRU (refreshed on a hit). A fault fills a free frame if one exists, else evicts the frame with the smallest stamp. It prints the frames and HIT/FAULT for each reference. The reference string is passed as one comma-separated argument because the xv6 shell allows only 10 arguments. xv6 has no floating point, so hit ratios use integer arithmetic with two decimals. Run: *pagerepl 3 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 [fifo|lru]* and *pagerepl -t ...* for the table.
Output: with 3 frames FIFO has 15 faults and LRU has 12, as expected. Table for the same string (20 references):

Frames	FIFO faults	FIFO hit ratio	LRU faults	LRU hit ratio
3	15	25.00%	12	40.00%
4	10	50.00%	8	60.00%
5	9	55.00%	7	65.00%

LRU is better at every size here. FIFO does not show Belady's anomaly on this string (faults fall as frames grow). Screenshots: q3_pagerepl_fifo_3frames.png, q3_pagerepl_lru_3frames.png, q3_pagerepl_table.png.